

HONEYWELL EMBEDDED ENGINEERING TEAMS — PA · IA · BA

AI Champions Programme

Module 03

Prompt Engineering Recap and Context Engineering

From prompt-only guesswork to structured, token-aware context — the discipline every SDD, MCP and PR-quality module from here depends on.



DURATION

2 Hours · Module 3 of 12



FORMAT

Guided Recap + Progressive Lab



DELIVERED FOR

Honeywell Embedded Engineering

How We'll Spend These Two Hours

Six connected moves — from a prompting recap to a measured, four-round progression.



01

Prompt Engineering Recap

15 min

Task framing, instructions, constraints, examples, decomposition, iterative prompting & verification.



02

Weak vs. Strong Prompt Walkthrough

15 min

See the same embedded task fail, then succeed, as prompting discipline improves.



03

The Embedded Context Inventory

20 min

What actually belongs in context: code, build, design & history — and what doesn't.



04

Context Layering & Compression

20 min

Selection, layering, task-scoped context, and avoiding context inflation.



05

Four-Round Progressive Lab

40 min

Solve one embedded task four times, adding structured context each round.



06

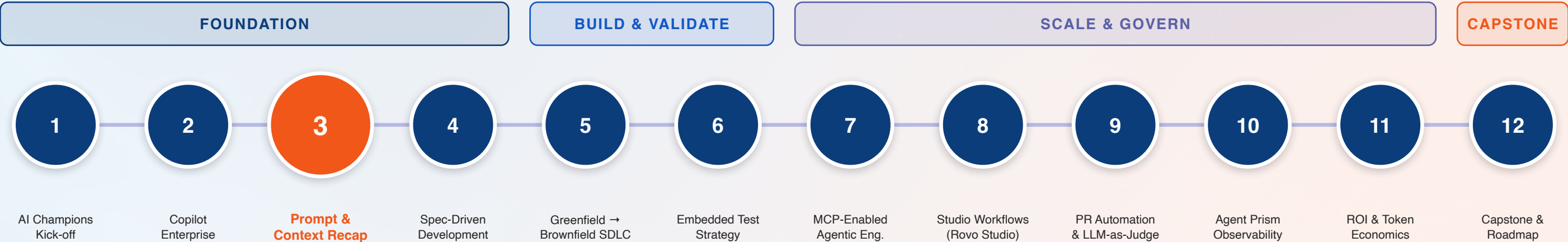
Measure & Compare

10 min

Chart output quality, token usage, iteration count & completion time across rounds.

Module 03 in the 12-Module Journey

The discipline layer — Module 04's Spec-Driven Development is context engineering applied to a full feature.



Why this matters: Spec-Driven Development in Module 04, brownfield repository archaeology in Module 05, and MCP context/state boundaries in Module 07 all assume you can scope, layer and compress context on purpose — not by accident.

Prompt Engineering Recap: The Six Building Blocks

Not new material — the discipline you already use, made explicit before we add context on top of it.



01

Task Framing

State the goal, the file(s) in scope, and what “done” looks like before asking for code.



02

Instructions

Explicit, ordered steps beat a single vague request — especially for multi-file changes.



03

Constraints

Say what must NOT change: timing behaviour, public API, existing test results.



04

Examples

Show one existing function in the target style; Copilot mirrors it far better than a description.



05

Decomposition

Break a large task into independently-verifiable steps — same discipline as breaking down a user story.



06

Iterative Prompting & Verification

Treat the first response as a draft. Refine, re-check against constraints, repeat.

From Prompt-Only to Context-Engineered

Same task, same engineer — the only variable is what surrounds the prompt.

PROMPT-ONLY

- Copilot sees only the open file — no header relationships, no build config, no design intent.
- Generated code often re-implements logic that already exists elsewhere in the repository.
- No traceability from requirement to acceptance criteria to implementation.
- High iteration count: each miss costs a full re-prompt with no accumulated context.
- Token cost concentrated in re-explaining the same background, session after session.



CONTEXT-ENGINEERED

- Copilot sees task-scoped source/header relationships, interfaces, and relevant design notes.
- Generated code matches existing conventions because those conventions were supplied as context.
- Requirement → constraint → acceptance criteria stays traceable through to implementation.
- Lower iteration count: the first draft already respects real interfaces and standards.
- Token cost spent once on layering context, then reused efficiently within the task's scope.

What the Calibration Survey Told Us About Context Discipline

Real answers from 19 Honeywell engineers — this cohort's instincts are better than “prompt-only” suggests.

Your First Move on an Existing Codebase

13

of 19

• Architecture first (13)

• Spec / impact first (5)

• Find similar code (1)

13 of 19 already work out architecture, interfaces & dependencies before writing anything — without being taught to. This module gives that instinct a name and a repeatable method.

But in practice today: 10 of 19 still hand that context to AI mainly through prompts alone — the good instinct doesn't yet reach the AI assistant.

How Do You Use AI on a Large C/C++ Repository?

Give it repo, architecture, design & test context, ask it to analyse first

8

Use structured specs, repo context, skills & agentic workflows

4

Hand it the relevant files and ask it to make the change

4

Ask it about individual functions or files

3

Signal for today's lab: 8 of 19 already default to the context-engineered pattern — the four-round lab turns that into a habit the whole cohort shares.

The Embedded Context Inventory

Nine context sources, four categories — what actually belongs in scope for a given task.



Code Context

Source / Header Relationships

Interfaces & Contracts



Build Context

Build Files

Configuration



Design Context

Hardware-Abstraction Boundaries

Design Notes

Coding Standards



History Context

Issue History

Tests

Task-scoped, not repository-wide: Pull only the inventory items relevant to the task at hand — the full list is a menu, never a checklist to supply every time.

Context Engineering Practices

Five disciplined moves that turn a pile of files into task-scoped, token-aware context.



Context Selection

Choose which inventory items this specific task actually needs — resist supplying everything “just in case.”



Layering

Build context in ordered layers — task, then repository, then design — rather than one flat dump.



Compression

Summarize long design docs and issue threads into the two or three facts that matter for this task.



Task-Scoped Boundaries

Draw an explicit line: these files are in scope, everything else is out — stated, not assumed.



Token-Aware Practices

Treat context like a budget. Every file you add has a cost; spend it where it changes the output.

Cohort signal: “Use tokens more efficiently” was a top-three training ask in the pre-training survey (7 of 19) — Practice 5 above is built for exactly that.

Context Layering in Action

Add layers only as the task demands them — each layer has a token cost.



Layer 1 — Task Framing

Goal, scope, and definition of done. Always present, always cheapest.

~200 tokens



Layer 2 — Code Context

Relevant source/header pairs & interfaces the task will actually touch.

~800 tokens



Layer 3 — Design Context

HAL boundaries, coding standards & design notes — add only for non-trivial changes.

~1,500 tokens



Layer 4 — History Context

Issue history & prior related tests — reserved for brownfield or regression-sensitive work.

~2,800+ tokens

The rule: Stop adding layers the moment the task is well-specified. A one-function bug fix rarely needs Layer 4 — a brownfield behaviour-preservation task almost always does.

Anti-Patterns: How Context Inflation Happens

Five habits that quietly turn task-scoped context into an expensive, noisy dump.

DO THIS

-  Scope context to the files this task touches
-  Summarize a long design doc into 2–3 relevant facts
-  Add history/issue context only for brownfield work
-  State an explicit in-scope / out-of-scope boundary
-  Re-use a compressed context layer across a session

NOT THAT

-  Paste the entire repository “just to be safe”
-  Attach the full design document unfiltered
-  Include full issue history for every task by default
-  Let Copilot infer scope from whatever's open
-  Re-explain the same background in every new prompt

Prompt Recap in Practice: Weak vs. Strong

Same defect, same engineer — the six building blocks made explicit.

WEAK PROMPT

"The CAN driver has a bug. Can you fix it?"

- ✗ No file or function named
- ✗ No definition of "bug" or expected behaviour
- ✗ No constraint on what must stay unchanged
- ✗ No example of the surrounding code style

STRONG PROMPT

"In can_driver.c, the retry counter in can_bus_off_recover() doesn't reset after a successful recovery (see #4021). Fix only that reset logic — don't change the retry timing constants. Match the error-handling style used in can_send()."

- ✓ Names the file, function & issue reference
- ✓ States exactly what must not change
- ✓ Points to an existing style example

Notice: The strong prompt isn't longer for its own sake — every added word maps directly to one of the six building blocks from Slide 4.

What We'll Measure

Four signals, captured every round — turning “it felt better” into a number.



Output Quality

Does the generated code compile, pass existing tests, and match acceptance criteria — scored 1–5.



Token Usage

Total tokens consumed for the round, including every re-prompt and context layer supplied.



Iteration Count

How many back-and-forth exchanges it took to reach an acceptable result.



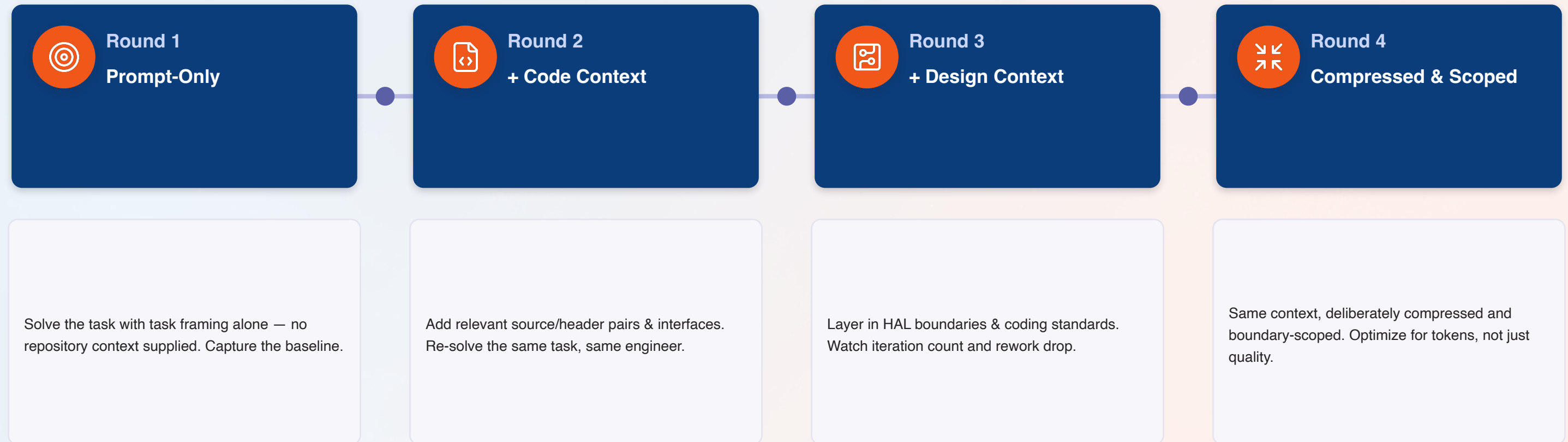
Completion Time

Wall-clock time from first prompt to an engineer-approved change, end to end.

Where this goes next: These same four signals reappear as the token/cost and ROI model in Module 11 — today's worksheet is the first data point.

Hands-On Lab: The Four-Round Progression

One embedded task, solved four times — each round adds exactly one layer from Slide 8.



Module 03 Outcomes & What's Next

Everything below should be true before we open GitHub Spec Kit in Module 04.



The six prompt-engineering building blocks are second nature, not a checklist



Practiced context selection, layering, compression & explicit scope boundaries



Ran the same task across four rounds — prompt-only through compressed & scoped



Understand why context inflation happens and the five habits that prevent it



Can name all nine items in the embedded context inventory, in four categories



Saw a weak prompt and a strong prompt solve the same defect, side by side



Captured output quality, token usage, iteration count & time for every round



Ready to treat a specification as the next, larger unit of context in Module 04



NEXT — MODULE 04: SPEC-DRIVEN DEVELOPMENT FOR EMBEDDED SOFTWARE

2 hours · Requirements, constraints, interfaces & acceptance criteria — spec → plan → tasks → implementation with GitHub Spec Kit / OpenSpec.

Questions & Discussion

Module 03 — Prompt Engineering Recap and Context Engineering